

stepperMotorController

1 Executive Summary

This module drives a stepper motor toward a reference angle by issuing integer step commands. Internally the controller runs a small state machine (OFF / IDLE / MOVING / STOPPING / SETTLING) that decides when to command a new move, when a changed reference should interrupt the current move, and how long to wait after a stop before accepting new commands. The controller always commands the shortest path around a full revolution, so a reference on the opposite side of the motor is reached via the smaller of the two directions.

2 Message Connection Descriptions

The following table lists all the module input and output messages. Message connections are set by the user from Python.

Table 1: Module I/O Messages

Msg Variable Name	Msg Type	Description
motorRefAngleInMsg	HingedRigidBodyMsgF32Payload	Reference motor angle input message (uses <code>theta</code> , [rad])
motorStepCommandOutMsg	MotorStepCommandMsgPayload	Commanded motor steps output message (uses <code>stepsCommanded</code>). Written only when a new MOVE command is issued.

3 Algorithm Parameters

The following table lists the parameters on the pure algorithm (`StepperMotorControllerAlgorithm`). The Xmera adapter re-exposes all of these parameters through same-named setters/getters that forward to the algorithm.

Table 2: Algorithm Parameters

Parameter Name	Type	Units	Default	Description	Bounds
stepsPerRevolution	int	[steps]	360	Number of motor steps per full revolution	Must be strictly positive (checked in setter)
settleCountMax	int	[ticks]	10	Number of control ticks to remain in SETTLING before returning to IDLE	Must be non-negative (checked in setter)
currentPositionToleranceint		[steps]	1	Tolerance between the current and target position used for the IDLE move trigger and the MOVING move-complete check	Must be non-negative (checked in setter)
desiredPositionToleranceint		[steps]	0	Tolerance between the commanded and desired position used in MOVING to detect a changed reference	Must be non-negative (checked in setter)

4 Module Parameters

The following table lists the parameters that live only on the Xmera adapter (**StepperMotorController**) — they configure the motor-motion simulator and the initial conditions applied on **reset()**. The adapter additionally re-exposes every algorithm parameter from the table above.

Table 3: Module Parameters (Xmera adapter only)

Parameter Name	Type	Units	Default	Description	Bounds
initialAngle	float	[rad]	0.0	Initial motor angle, used to seed the tracked current position on reset	N/A
controlFrequency	float	[Hz]	1.0	Rate at which updateState() is called by the simulation task	Must be strictly positive (checked in setter)
motorFrequency	float	[Hz]	1.0	Maximum motor step rate (steps per second) used by the Xmera motor-motion simulator	Must be strictly positive (checked in setter)

5 Algorithm Input/Output

The following tables list the inputs and outputs of the pure algorithm **update()** method, independent of the Xmera messaging layer and the motor-motion simulation.

Table 4: Algorithm Inputs

Variable	Type	Units	Description
currentPosition	int	[steps]	Current motor step position, tracked by the caller
referenceAngle	float	[rad]	Reference motor angle; converted to steps internally via angleToSteps()
isMotorMoving	bool	[-]	Indicates whether the motor is currently moving (used to gate the STOPPING → SETTLING transition)

Table 5: Algorithm Outputs (StepperMotorControllerOutput)

Variable	Type	Units	Description
commandType	StepperMotorCommandType	[-]	NONE , MOVE , or STOP
stepsToMove	int	[steps]	Wrapped (shortest-path) step delta; valid only when commandType == MOVE

6 Algorithm Description

The stepper motor controller operates in integer step counts. Angles are converted to steps using the motor's fixed step size,

$$n = \text{round}\left(\theta \frac{N}{2\pi}\right)$$

where θ is an angle in radians and N is `stepsPerRevolution`. The controller never reasons in radians internally: both the current and desired positions are integer step counts.

6.1 Shortest-Path Wrapping

To always command the shortest rotation, step deltas are wrapped into the interval $[-\frac{N}{2}, +\frac{N}{2}]$ via

$$\text{wrap}(\Delta) = \begin{cases} \Delta - N & \text{if } \Delta > \frac{N}{2} \\ \Delta + N & \text{if } \Delta < -\frac{N}{2} \\ \Delta & \text{otherwise} \end{cases}$$

applied after taking $\Delta \bmod N$. A motor at -170° asked to move to $+170^\circ$ therefore commands a 20° forward step, not 340° backward.

6.2 State Machine

Let n_c be the caller-supplied `currentPosition`, n_d the desired position (derived from the reference angle), and n_m the position most recently commanded (stored internally). Let τ_c be the `currentPositionTolerance` and τ_d the `desiredPositionTolerance`.

- **OFF** — no transitions; the algorithm stays in this state until the caller reassigns it (used to hold the motor quiescent).
- **IDLE** — the motor is quiescent. If $|\text{wrap}(n_d - n_c)| > \tau_c$, the algorithm emits a **MOVE** with `stepsToMove` = `wrap(n_d - n_c)`, stores $n_m := n_d$, and transitions to **MOVING**.
- **MOVING** — the motor is executing a commanded move. Two conditions are checked each tick:
 1. *Move complete.* If $|\text{wrap}(n_m - n_c)| \leq \tau_c$, the algorithm transitions to **STOPPING** (no **STOP** command issued; the motor is already at target).
 2. *Reference changed.* If $|\text{wrap}(n_m - n_d)| > \tau_d$, the algorithm emits a **STOP** command and transitions to **STOPPING**. The caller is expected to halt the motor at its current position; the controller will re-plan from that position once it returns to **IDLE**.
- **STOPPING** — the motor is decelerating. The algorithm transitions to **SETTLING** and resets the settle counter as soon as the caller reports `isMotorMoving == false`. In Xmera simulations the caller leaves `isMotorMoving` at `false`, so this transition is immediate.
- **SETTLING** — a counter runs for up to `settleCountMax` ticks. Once the counter reaches the limit the algorithm returns to **IDLE** and is ready to issue the next move.

The two tolerances are deliberately separated:

- `currentPositionTolerance` governs whether the caller's physical position is close enough to the target to count as "at target" — tuned to the motor's step-level resolution and positioning repeatability.
- `desiredPositionTolerance` governs whether a new reference is different enough from the currently-commanded target to be worth interrupting the in-progress move — tuned to avoid interrupting on noise in the reference signal.

6.3 Motor-Motion Simulation (Xmera Adapter)

The algorithm itself does not simulate the motor. The Xmera adapter advances the tracked current position toward the commanded position each tick using a fractional-step accumulator. Given `motorFrequency` f_m and `controlFrequency` f_c , each tick adds f_m/f_c to a running accumulator; the integer part is taken as the number of whole steps advanced that tick and the fractional remainder carries over. This produces the expected patterns for non-integer ratios (e.g. a 1.5 ratio alternates 1, 2, 1, 2, ...). The advance never overshoots the commanded position.

7 Algorithm Assumptions and Limitations

- Motor motion is assumed to be quantized at the step level; reference angles that fall between steps are rounded to the nearest step.
- The caller is responsible for tracking the motor's current position and passing it to `update()` each tick.
- When a **STOP** is issued mid-move, the Xmera adapter freezes the commanded position at the current step; any remaining steps from the prior **MOVE** are discarded.
- The **OFF** state is entered only via caller intervention; there is no transition into **OFF** from within the state machine.

8 Module Description (Xmera Usage)

The `StepperMotorController` Xmera adapter provides the simulation integration layer for the algorithm. It:

- Reads `motorRefAngleInMsg` to obtain the reference angle each tick.
- Tracks the motor's current step position, the commanded position, and a fractional-step accumulator.
- On `reset()`, seeds the current and commanded positions from `initialAngle` (converted to steps).
- On each `updateState()`, calls the algorithm with the tracked current position and the reference angle.
- On a **MOVE** output, writes `motorStepCommandOutMsg` with the commanded step delta and sets the new commanded position.
- On a **STOP** output, freezes the commanded position at the current position.
- Advances the tracked current position toward the commanded position via the fractional-step accumulator.

9 User Guide

Typical usage in Python is:

```
module = stepperMotorControllerF32.StepperMotorController()
module.modelTag = "stepperMotorController"
module.stepsPerRevolution = 360
module.initialAngle = 0.0                # [rad]
module.controlFrequency = 10.0           # [Hz]
module.motorFrequency = 100.0           # [Hz]
module.settleCountMax = 2
module.currentPositionTolerance = 0
module.desiredPositionTolerance = 0

module.motorRefAngleInMsg.subscribeTo(ref_msg)
```

The commanded step delta is available on `motorStepCommandOutMsg` each time a new **MOVE** is issued.

`currentPositionTolerance` controls how close the motor must be to the target before the algorithm considers a move complete, and also how close an incoming reference must be to the current position before the algorithm declines to issue a move. `desiredPositionTolerance` separately controls how large a change in the reference angle must be, relative to the currently-commanded target, before the algorithm interrupts an in-progress move.